



AI & Software Engineering Workshop

Week 1 — Day 4: Memory and State

Edik Simonian

The problem

1. Tell your bot: *"my favorite color is blue"*
2. Ask: *"what's my favorite color?"* — it knows ✓
3. Restart the bot. Ask again.
4. **It has no idea who you are.**

Storage not configured – running in stateless mode — today we fix that line.

Key-value store = lockers

- A wall of lockers 
- **Key** = the locker number → `chat:12345`
- **Value** = whatever you put inside → the conversation
- `get(key)` / `set(key, value)` — that's the whole API
- Ours also has **TTL**: lockers that empty themselves after 30 days

Redis, DynamoDB, memcached — same idea. Ours is SQLite: a database in a single file.

Turn it on

`.env` :

```
SQLITE_PATH=./bot.db
```

- Restart → tell it your favorite color → restart again → ask
- **It remembers.** The file `bot.db` appeared — that's the memory
- No signup, no server, no credit card. It's just a file
- Delete the line → graceful fallback to stateless mode (try it)

Live-code: `/remember` and `/recall`

```
@bot.message_handler(commands=["remember"], func=is_allowed)
def cmd_remember(message):
    note = message.text.split(maxsplit=1)[1] if " " in message.text else ""
    store.set(f"note:{message.from_user.id}", note)
    bot.send_message(message.chat.id, "Saved!")
```

`/recall` is the mirror image: `store.get(f"note:{...}")` .

Solo build: a full notes feature

- `/remember <text>` — add a note (not replace!)
- `/recall` — list **all** saved notes
- `/forget` — clear them

Hint: the store saves **strings**. To keep a list, `json.dumps` it on the way in, `json.loads` it on the way out.

Production touches already in the repo

- **Rate limit** — `RATE_LIMIT` env var, default 250 messages/user/day
- **History trimming** — last 20 messages per user, 30-day expiry
- **Whitelist** — `ALLOWED_USERS` makes the bot **silent** to strangers
(silence, not rejection — scanners learn nothing)
- **Dedupe** — Telegram retries webhooks; the bot won't answer twice

All built on the same store you just used.

Debug challenge 🐛

The instructor just broke something in your bot.

- Symptom first: what *exact/y* stopped working?
- `print()` is a debugger — trace the path of one message
- Found it? Don't fix it yet — explain it to your neighbor first

Today → Tomorrow

Today your bot remembers — conversations, notes, limits — all in one SQLite file.

Tomorrow: the internet. Your bot moves to a real server and answers while your laptop sleeps.